rsa

第二次解密脚本 Python

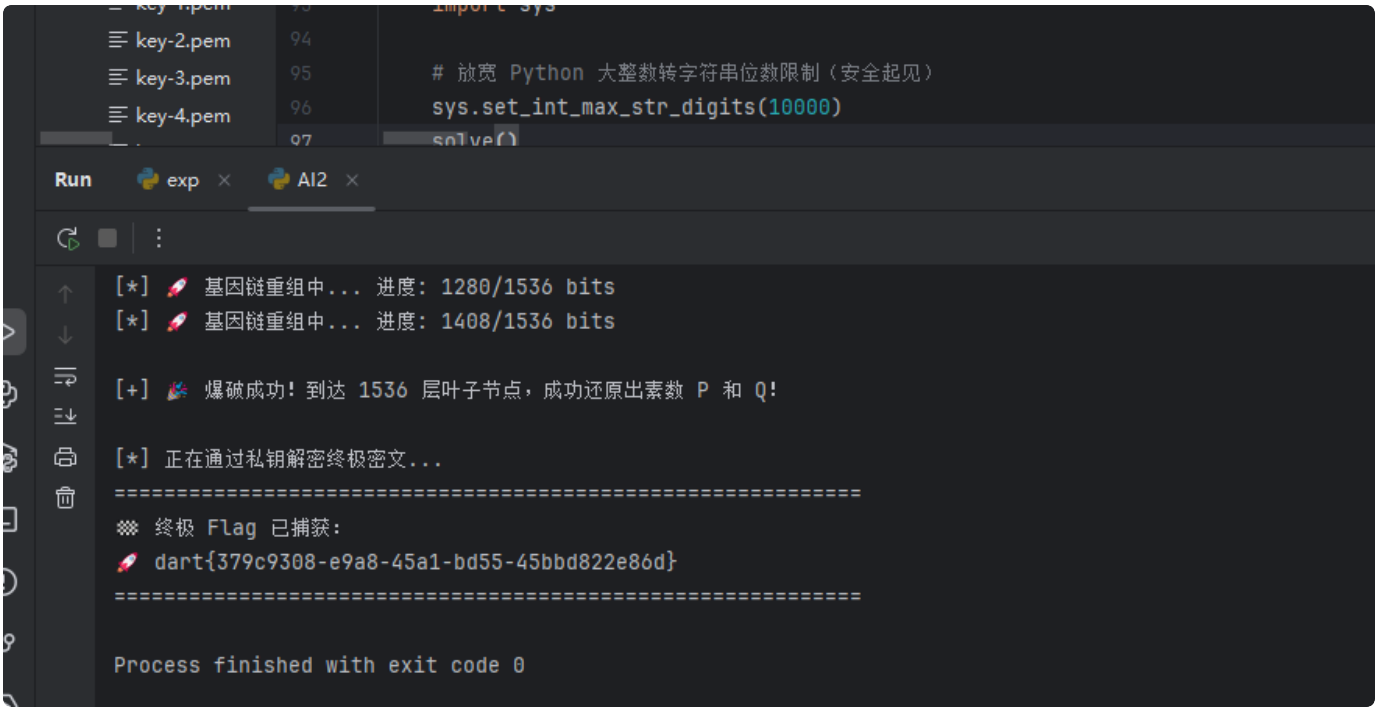
```python
import math
import random
import hashlib

# 题目提供的参数
n = 995733630482752347642314027694641164160870100149923192212010939056874399336324304660679920370461207121995652504821970043013433419606553579445773308854709184660077305707186480251435616563957515184286307425870232674506338246369369535248687352636660894523484660181950994715358239693650071206805465929990221957 81
e = 120768305392951935330332122324875688882009631230241892876294934800586382221469724961108143728838297656926231071911293061907889767042505023162654399968917641014470171903770149802935897974030952495383915349866389730352859008675484201922112411637789190289215023057909798803460504288391028740860466228332119132 99

def get_convergents(e, n):
    """求 e/n 的连分数渐进分数"""
    a, b = e, n
    cf = []
    while b:
        cf.append(a // b)
        a, b = b, a % b

    n0, d0 = 0, 1
    n1, d1 = 1, 0
    for q in cf:
        n2, d2 = q * n1 + n0, q * d1 + d0
        yield n2, d2
        n0, d0, n1, d1 = n1, d1, n2, d2


def factor_from_multiple(n, M, tries=30):
    """利用拉宾-米勒式的二次探测分解N"""
    t, s = M, 0
    while t > 0 and t % 2 == 0:
        t //= 2
        s += 1

    for _ in range(tries):
        a = random.randint(2, n - 2)
        x = pow(a, t, n)
        if x in (1, n - 1):
            continue
```

```python
        for _ in range(1, s):
            y = pow(x, 2, n)
            if y == 1:
                p = math.gcd(x - 1, n)
                if p > 1 and n % p == 0:
                    return p, n // p
            if y == n - 1:
                break
            x = y
    return None, None


def solve():
    print("[*] 正在执行 Wiener's Attack 连分数分析 (包含约分补偿)...")

    # 穷举因为使用 lambda(n) 而被约分掉的常见乘数
    mults = [1, 2, 3, 4, 5, 6, 8, 10, 12, 16, 20, 24, 32, 40, 48, 64, 80, 96, 128, 160, 192, 256, 384, 512, 768, 1024]

    for k_i, d0 in get_convergents(e, n):
        if d0 == 0: continue
        if d0.bit_length() > 300: break  # Wiener 攻击的私钥 d 必定比较小

        cands = set()
        for g in mults:
            cands.add(d0 * g)
            if d0 % g == 0:
                cands.add(d0 // g)

        for d_cand in cands:
            M = e * d_cand - 1
            if M <= 0 or M % 2 != 0:
                continue

            # 快速探测: 判断 2^M ≡ 1 (mod n)
            if pow(2, M, n) == 1:
                p, q = factor_from_multiple(n, M)
                if p and q:
                    print(f"\n[+] 🎉 成功分解 N !")
                    print(f"    p = {p}")
                    print(f"    q = {q}")

                    # 计算 SHA256 密码
                    pass_str = str(p + q)
                    flag_hash = hashlib.sha256(pass_str.encode()).hexdigest()

                    print("\n" + "=" * 60)
```

```python
                    print(f"🔑 Level 3 压缩包密码为: \n{flag_hash}")
                    print("=" * 60)
                    return


if __name__ == "__main__":
    solve()
```

第三层，常规的 RSA 加密，外加一个极度复杂的位运算等式 leak，里面全是 p和q 的移位、异或、加法。加法和位运算都有一个核心特性：低位的结果绝不受高位影响，用深度优先搜索从地位往上猜

脚本解出最后flag `dart{379c9308-e9a8-45a1-bd55-45bbd822e86d}`



第三次解密脚本 Python

```python
from Crypto.Util.number import long_to_bytes

# ----------------- 题目提供的数据 -----------------
n = 36565431707806713021023697858213189485215332322595980297463970611080068184680536762916341127876111765549243536289724824717545191937172323138488477445222155922819211472978988923074456743352499531744980259044938555308927856692816222280673288555502224572907049911864045112943924286269010716685405173911325566328888646946533348535577640277494811994169018813323076609664629570164888840470470462025195205081024616632463284379308952340747766544599678578432073205301701440230567822059289480505199198254775625145944490699640987943220051569208398486154817171846155814714711051673108777841076538269488018380839370609291033069520847869828342421198770462192608409661429972646760145751042311223497708829748184275915385517199902203473456143996396432576855913215006484374020849194673460496838420429939756964477110802895590639592710450825069685321034452416379717341730372243941039441536923100480436935028707062253197879022312189545484120182591
e = 65537
c = 17579146686041540897017104469074457875123465003782592246589479232172729442112147574887350534842139170676987150500104521934635987109891230208152958147095187427558203833640976959295493664142234212425998407554413117718359824314390739323403563416363468824640584934594550916916530778477767716315604989305895699886466132189102311536100317492871716491529229290668286056555704316564260742372612555611294328893187002348848573538914027337918361554960848250678780590017236176908729123594711098886648017930791931444893234555963417086979111589425056117099462521016704507965503130791395602818436120456815459926269448032308327767944543536391225951076712678592922228613673261214351548626075178903299256213679926677288998784220371828178606415301462347301966332373399017265089067338975561467515030971276727181929586427763896919406713563673041828254335925778998814448150625811633869470758872185378024830457568860194267498557237151929816359719431
leak = 15333802221058597068749544440922796126178374957011499393123131742763432111830960057590366267828669807196230443693137197717919726606344761630447746220652834200815126461104098287385958362823475501375700308238256201221917507095782215494423112622840334104747768665237152395102807122171950309564641353084290895207161051853000596788006852670156447223768609504348129620154316170164416015171264901405200201211682911039481158687355926676333906917249570492290665149124700105709531471870963493718761989055008600970673771251553207 6

# 常量
C1 = 0xDEADBEEFCAFEBABE123456789ABCDEFFEDCBA9876543210
C2 = 0xCAFEBABEDEADBEEF123456789ABCDEF0123456789ABCDEF
C3 = 0x123456789ABCDEFFEDCBA9876543210FEDCBA987654321
MASK64 = (1 << 64) - 1
MASK128 = (1 << 128) - 1
```

```python

# 因为 python 的 + 优先级高于 ^，等式可以化简为：
# leak = ( A + B ) ^ ((p*q) & MASK64)
# 而 (p*q) = n，所以尾部是个常数。
B_target = leak ^ (n & MASK64)


def solve():
    print("[*] 启动 DFS 状态空间树进行 2-adic 逐位推导 (LSB Lifting)...")

    # 栈结构：(当前处理的位数 k，猜测的 p 值，猜测的 q 值)
    # 素数都是奇数，所以最低位第 0 位必为 1，直接从第 1 位开始猜测
    stack = [(1, 1, 1)]

    p_final, q_final = 0, 0
    max_k_print = 0

    while stack:
        k, p_val, q_val = stack.pop()

        # 已经推导满 1536 位，开始验证并输出
        if k == 1536:
            if p_val * q_val == n:
                p_final, q_final = p_val, q_val
                print(f"\n[+] 🎉 爆破成功！到达 1536 层叶子节点，成功还原出素数 P 和 Q！")
                break
            continue

        if k > max_k_print and k % 128 == 0:
            print(f"[*] 🚀 基因链重组中... 进度: {k}/1536 bits")
            max_k_print = k

        # 扩充模数掩码，用于截断判断
        mod = 1 << (k + 1)
        rhs_n = n & (mod - 1)
        rhs_b = B_target & (mod - 1)

        # 枚举 p 和 q 在当前位 (第 k 位) 分别是 0 还是 1
        for bp in (0, 1):
            p_next = p_val | (bp << k)

            for bq in (0, 1):
                q_next = q_val | (bq << k)

                # 【剪枝条件 1】检查当前组装的 p 和 q 乘积的低 k+1 位，是否吻合 n 的低 k+1 位
                if (p_next * q_next) & (mod - 1) != rhs_n:
```

```python
                    continue

                # 【剪枝条件 2】校验 leak 等式的低 k+1 位运算结果是否吻合
                A = ((p_next * C1) ^ (q_next * C2) ^ ((p_next & q_next) << 64) ^ ((p_next | q_next) << 48) ^ (
                            (p_next ^ q_next) * C3))
                s128 = (p_next + q_next) & MASK128

                # A + s128 的低 k+1 位必须等于目标的低 k+1 位
                if ((A + s128) & (mod - 1)) != rhs_b:
                    continue

                # 全部校验通过，这一分支暂时合法，压入栈中供下一轮探索
                stack.append((k + 1, p_next, q_next))

    # 计算 RSA 私钥 d 并解密获取最终 Flag
    if p_final and q_final:
        print("\n[*] 正在通过私钥解密终极密文...")
        phi = (p_final - 1) * (q_final - 1)
        d_real = pow(e, -1, phi)
        m = pow(c, d_real, n)

        flag = long_to_bytes(m).decode(errors='ignore')

        print("=" * 60)
        print("🏁 终极 Flag 已捕获：")
        print(f"🚀 {flag}")
        print("=" * 60)
    else:
        print("[-] 推导失败，未能还原出正确的 P 和 Q。")


if __name__ == "__main__":
    import sys

    # 放宽 Python 大整数转字符串位数限制（安全起见）
    sys.set_int_max_str_digits(10000)
    solve()
```

dart{379c9308-e9a8-45a1-bd55-45bbd822e86d}